

Búsqueda Heurística

Javier Béjar

Inteligencia Artificial - 2025/2026 1Q

CS - GEI

Búsqueda Heurística

- ⊙ Supone la existencia de una función de evaluación que debe medir la distancia estimada al (a un) objetivo ($h(n)$)
- ⊙ Esta función de evaluación se utiliza para guiar el proceso haciendo que en cada momento se seleccione el estado o las operaciones más prometedores
- ⊙ No siempre se garantiza encontrar una solución (de existir ésta)
- ⊙ No siempre se garantiza encontrar la solución más próxima (la que se encuentra a una distancia, número de operaciones menor)
- ⊙ Existen múltiples algoritmos:
 - Branch and Bound, Best First Search
 - A, A*
 - IDA*
 - Búsqueda local (Hill climbing, Simulated annealing, Alg. Genéticos)

Ramificación y acotación

- ⊙ Generaliza BFS, DFS
- ⊙ Se guarda para cada estado el coste (hasta el momento) de llegar desde el estado inicial a dicho estado. Guarda el coste mínimo global hasta el momento
- ⊙ Deja de explorar una rama cuando su coste es mayor que el mínimo actual
- ⊙ Si el coste de los nodos es uniforme equivale a una búsqueda por niveles

Algoritmo: Greedy Best First

Est_abiertos.insertar(Estado inicial)

Actual ← Est_abiertos.primer()

mientras no es_final?(Actual) **y** no Est_abiertos.vací?() **hacer**

 Est_abiertos.borrar_primer()

 Est_cerrados.insertar(Actual)

 hijos ← generar_sucesores (Actual)

 hijos ← tratar_repetidos (Hijos, Est_cerrados, Est_abiertos)

 Est_abiertos.insertar(Hijos)

 Actual ← Est_abiertos.primer()

- ⊙ La estructura de abiertos es una cola con prioridad
- ⊙ La prioridad la marca la función de estimación (coste del camino que falta hasta la solución)
- ⊙ En cada iteración se escoge el nodo más cercano a la solución (el primero de la cola), esto provoca que no se garantice la solución óptima

Operaciones:

- situar un bloque libre en la mesa
- situar un bloque libre sobre otro bloque libre

Heurístico 1:

- sumar 1 por cada bloque que esté colocado sobre el bloque que debe
- restar 1 si el bloque no está colocado sobre el que debe

Heurístico 2:

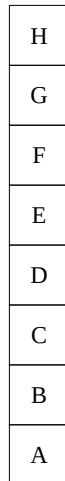
- si la estructura de apoyo es correcta sumar 1 por cada bloque de dicha estructura
- si la estructura de apoyo no es correcta restar 1 por cada bloque de dicha estructura



Estado Inicial

H1=4

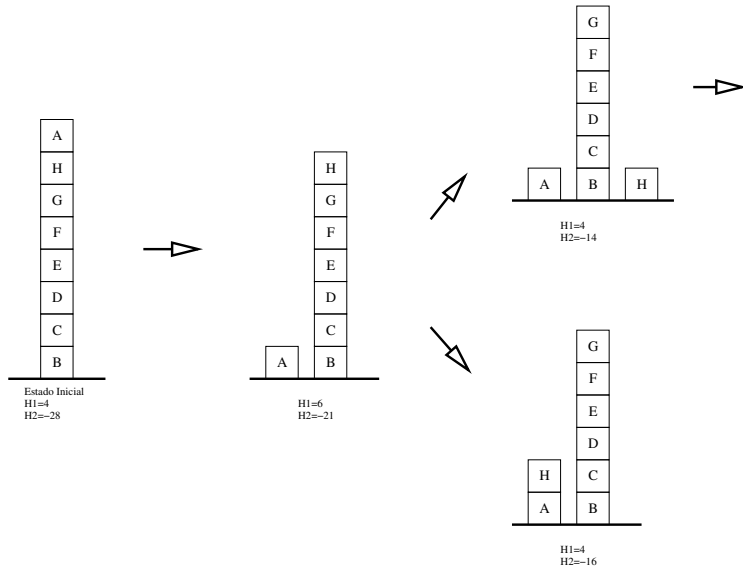
H2=-28



Estado Final

H1=8

H2= 28 (= 7+6+5+4+3+2+1)



2	8	3
1	6	4
7		5

Estado Inicial

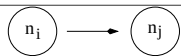
1	2	3
8		4
7	6	5

Estado Final

Posibles heurísticos (estimadores del coste a la solución)

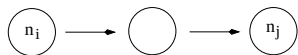
- ⊙ $h(n) = w(n) = \# \text{desclasificados}$
- ⊙ $h(n) = p(n) = \text{suma de distancias a la posición final}$
- ⊙ $h(n) = p(n) + 3 \cdot s(n)$

donde $s(n)$ se obtiene recorriendo las posiciones no centrales y si una ficha no va seguida por su sucesora sumar 2, si hay ficha en el centro sumar 1



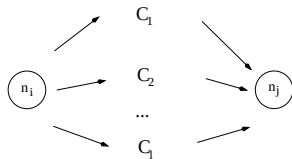
Coste de un arco

$$c(n_i, n_j) > 0$$



Coste de un camino

$$C = \sum_{x=i}^{j-1} c(n_x, n_{x+1})$$



Coste del camino mínimo

$$K(n_i, n_j) = \min_{k=1}^l C_k$$

Si n_j es un nodo terminal

$$h^*(n_i) = K(n_i, n_j)$$

Si n_i es un nodo inicial

$$g^*(n_j) = K(n_i, n_j)$$

Si existen varios nodos terminales $T = \{t_1, \dots, t_l\}$

$$h^*(n_i) = \min_{k=1}^l K(n_i, t_k)$$

Si existen varios nodos iniciales $S = \{s_1, \dots, s_l\}$

$$g^*(n_j) = \min_{k=1}^l K(s_k, n_j)$$

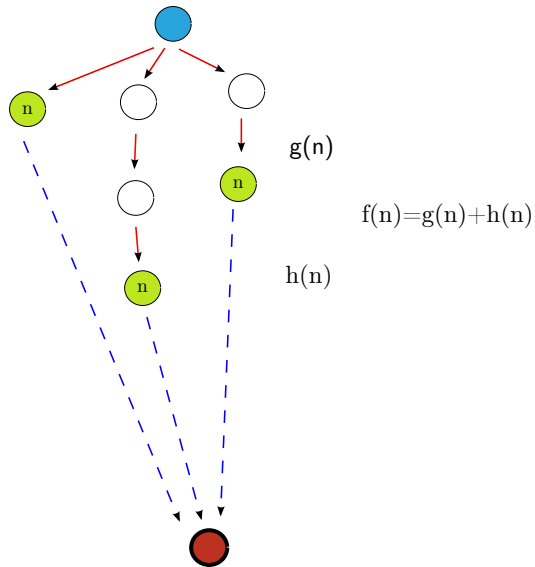
La función de evaluación tiene dos componentes:

1. coste para ir desde el (un) inicio al nodo actual
2. coste (estimado) para ir desde el nodo actual a una solución

$$f(n) = g(n) + h(n)$$

- ⊙ f es un valor **estimado** del coste total del camino que pasa por n
- ⊙ h (heurístico) es un valor **estimado** de lo que falta para llegar desde n al (a un) objetivo
- ⊙ g es un coste **real** (lo gastado por el camino más corto **conocido** hasta n)

La **preferencia** es siempre del nodo con menor f , en caso de empate, la preferencia es del nodo con menor h .



Con esta función podemos variar el comportamiento del algoritmo

- ⊙ Si $\forall n \ h(n) = 0$, todo estará controlado por g (estaremos en presencia de un algoritmo de Branch & Bound)
- ⊙ Si $\forall n \ h(n) = 0$ y además el coste de todos los arcos es 1 estaremos realizando una búsqueda en anchura. Si dicho coste fuera 0, la búsqueda sería aleatoria
- ⊙ Al ser h una estimación del verdadero coste h^* , cuanto más se aproxime h a h^* mayor será la tendencia a explorar en profundidad. Si $h = h^*$ entonces A^* converge directamente hacia el objetivo

Se puede demostrar que si $h(n)$ es un minorante del coste real $h^*(n)$, es decir si $\forall n \ h(n) \leq h^*(n)$ A^* encontrará (de haberlo) un camino óptimo.

Algoritmo: A*

Est_abiertos.insertar(Estado inicial)

Actual $\leftarrow$ Est_abiertos.primer()

mientras no es_final?(Actual) y no Est_abiertos.vací() hacer

 Est_abiertos.borrar_primer()

 Est_cerrados.insertar(Actual)

 hijos $\leftarrow$ generar_sucesores (Actual)

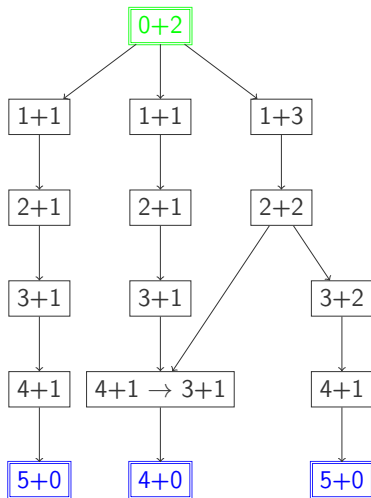
 hijos $\leftarrow$ tratar_repetidos (Hijos, Est_cerrados, Est_abiertos)

 Est_abiertos.insertar(Hijos)

 Actual $\leftarrow$ Est_abiertos.primer()

- ⊙ La estructura de abiertos es una cola con prioridad
- ⊙ La prioridad la marca la función de estimación ($f(n) = g(n) + h(n)$)
- ⊙ En cada iteración se escoge el mejor camino (el primero de la cola)
- ⊙ ¡Es el mismo algoritmo que el Best First!

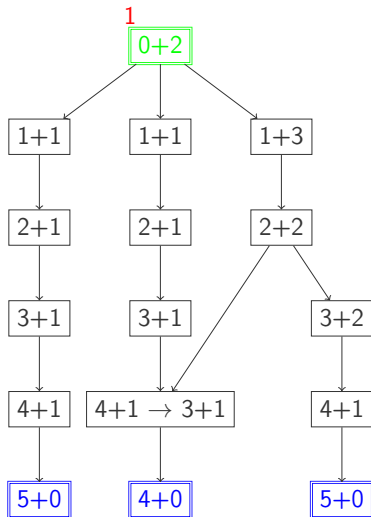
- ⊙ Si es un repetido que está en la estructura de abiertos
 - Si su coste es menor sustituimos el coste por el nuevo, esto podrá variar su posición en la estructura de abiertos
 - Si su coste es igual o mayor nos olvidamos del nodo
- ⊙ Si es un repetido que esta en la estructura de cerrados
 - Si su coste es menor reabrimos el nodo insertándolo en la estructura de abiertos con el nuevo coste **¡Atención!** No hacemos nada con sus sucesores, ya se reabrirán si hace falta
 - Si su coste es mayor o igual nos olvidamos del nodo



Abiertos

0+2

Cerrados

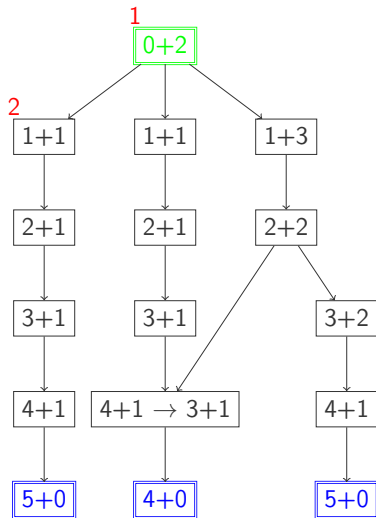


Abiertos

 $1+1$ $1+1$ $1+3$

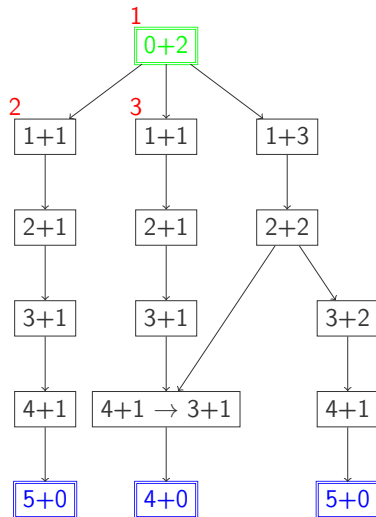
Cerrados

 $0+2$



Abiertos

Cerrados

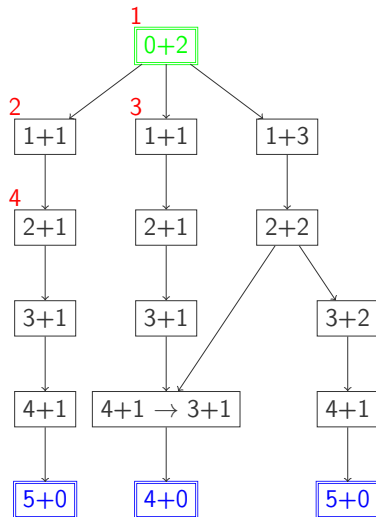


Abiertos

 $2+1$ $2+1$ $1+3$

Cerrados

 $0+2$ $1+1$ $1+1$

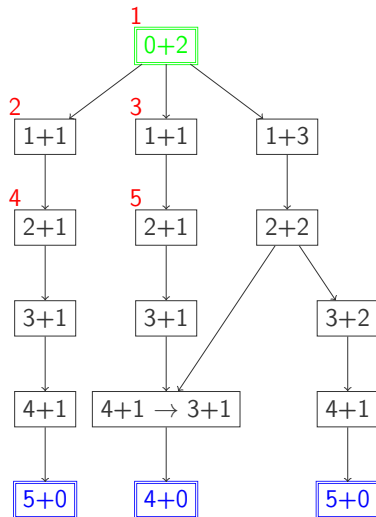


Abiertos

 $2+1$ $3+1$ $1+3$

Cerrados

 $0+2$ $1+1$ $1+1$ $2+1$

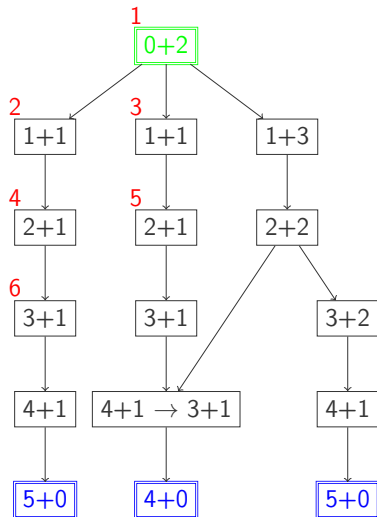


Abiertos

 $3+1$ $3+1$ $1+3$

Cerrados

 $0+2$ $1+1$ $1+1$ $2+1$ $2+1$

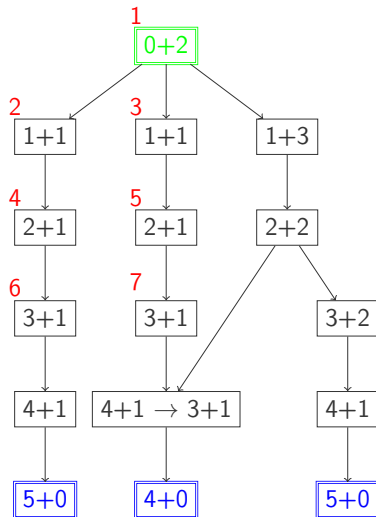


Abiertos

 $3+1$ $1+3$ $4+1$

Cerrados

 $0+2$ $1+1$ $1+1$ $2+1$ $2+1$ $3+1$



Abiertos

1+3

4+1

4+1

Cerrados

0+2

1+1

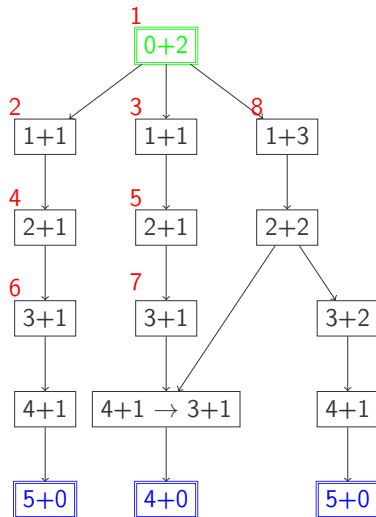
1+1

2+1

2+1

3+1

3+1

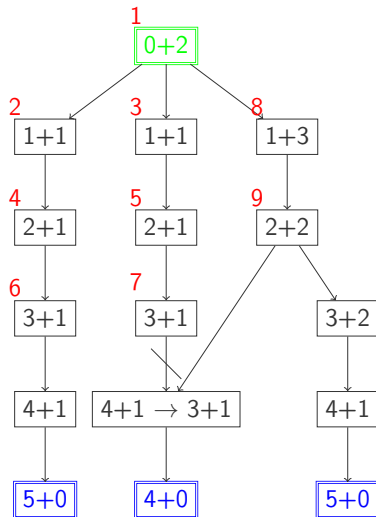


Abiertos

 $2+2$ $4+1$ $4+1$

Cerrados

 $0+2$ $1+1$ $1+1$ $2+1$ $2+1$ $3+1$ $3+1$ $1+3$

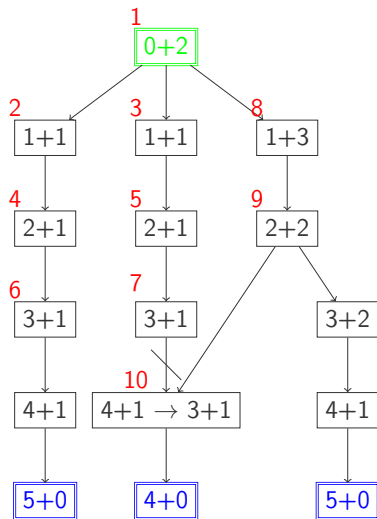


Abiertos

$3+1$
 $4+1$
 $3+2$

Cerrados

$0+2$
 $1+1$
 $1+1$
 $2+1$
 $2+1$
 $3+1$
 $3+1$
 $1+3$
 $2+2$



Abiertos

4+0

4+1

3+2

Cerrados

0+2

1+1

1+1

2+1

2+1

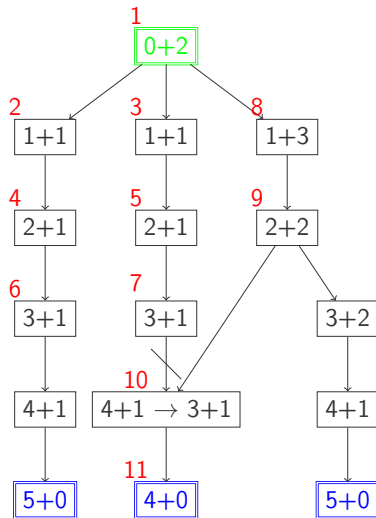
3+1

3+1

1+3

2+2

3+1



Abiertos

4+1

3+2

Cerrados

0+2

1+1

1+1

2+1

2+1

3+1

3+1

1+3

2+2

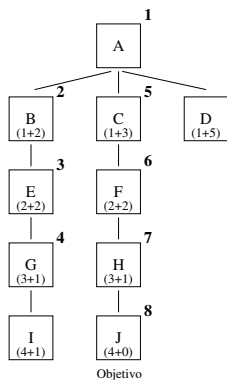
3+1

4+0

- ⊙ El algoritmo A^* encontrará la solución óptima dependiendo del heurístico
- ⊙ Si el heurístico es admisible la **optimalidad** está asegurada
- ⊙ Un heurístico es admisible si se cumple la siguiente propiedad

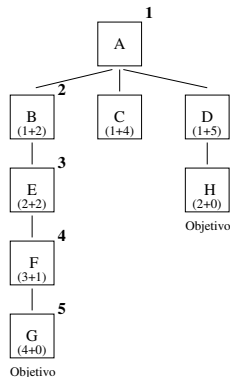
$$\forall n \quad 0 \leq h(n) \leq h^*(n)$$

- ⊙ Por lo tanto, $h(n)$ ha de ser un **estimador optimista**, nunca ha de sobreestimar $h^*(n)$



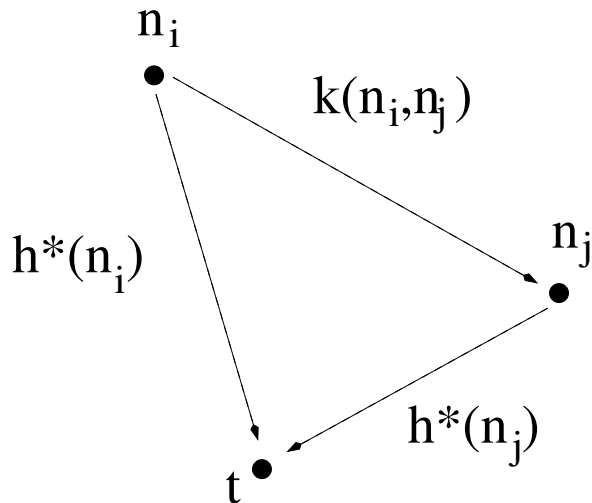
h subestima h^*

$h(B) = 2$, que no es real, hemos perdido tiempo pero acabaremos explorando otro camino (C)



h sobreestima h^*

si partiendo de D existiera un camino directo más corto nunca lo alcanzaríamos



- ⊙ $h^*(n_i) = \text{coste mínimo desde } n_i \text{ a } t$
- ⊙ $h^*(n_j) = \text{coste mínimo desde } n_j \text{ a } t$
- ⊙ $h^*(n)$ cumple desigualdad triangular

$$h^*(n_i) \leq k(n_i, n_j) + h^*(n_j)$$

- ⊙ La condición de consistencia exige que $h(n)$ se comporte como $h^*(n)$

$$h(n_i) - h(n_j) \leq k(n_i, n_j)$$

- ⊙ $h(n)$ consistente $\implies h(n)$ estimador uniforme de $h(n)$

- ⊙ $h(n)$ consistente $\implies g(n) = k(s, n)$ (camino óptimo a n)
- ⊙ Si $g(n) = k(s, n)$, dado que $h(n)$ siempre es constante $\implies f(n)$ es mínima
- ⊙ En este caso **no es necesario tratar los nodos duplicados cerrados** ya que los nodos expandidos ya no se podrán reexpandir (hemos llegado a ellos por el camino mínimo)

Dado un problema, existen tantos A^* para resolverlo como estimadores podamos definir.

Más informado

Para h_1 y h_2 admisibles, si se cumple

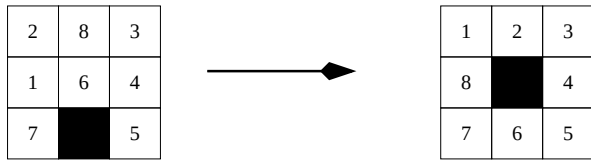
$$\forall n \neq final \quad 0 \leq h_2(n) < h_1(n) \leq h^*(n)$$

Entonces A_1^* es más informado que A_2^*

- ⊙ Si A_1^* es más informado que A_2^* entonces si el nodo n es expandido por $A_1^* \implies n$ es expandido por A_2^* (pero no al revés)
- ⊙ Eso quiere decir que A_1^* expande menor número de nodos que A_2^*

- ⊙ ¿Siempre elegiremos algoritmos más informados?
- ⊙ Compromiso entre:
 - Tiempo de cálculo de h
 - $h_1(n)$ requerirá más tiempo de cálculo que $h_2(n)$
 - Número de reexpansiones
 - A_1^* puede que reexpanda más nodos que A_2^*
 - Pero si A_1^* es consistente seguro que no lo hará
- ⊙ Perdida de admisibilidad
 - Puede interesar trabajar con no admisibles para ganar rapidez
 - Algoritmos A_ϵ^* (ϵ -admisibilidad)

Ocho puzzle

 h_0

$h_0(n) = 0$ Equivalente a anchura prioritaria, h_0 admisible, muchas generaciones y expansiones

 h_1

$h_1(n) = \# \text{piezas mal colocadas}$ h_1 admisible, h_1 mas informado que h_0

h_2

$$h_2(n) = \sum_{i \in [1,8]} d_i \quad d_i \equiv \begin{array}{l} \text{distancia entre posición de la pieza} \\ i \text{ y su posición final} \end{array}$$

h_2 admisible, h_2 no más informado que h_1

$$h_2([1, 2, 3, 4, 5, 6, 7, \emptyset, 8]) = h_1([1, 2, 3, 4, 5, 6, 7, \emptyset, 8]) = 1$$

 h_3

$$h_3(n) = \sum_{i \in [1,8]} d_i + 3 \cdot S(n) \quad ; \text{ con } S(n) = \sum_{i \in [1,8]} s_i$$

$$s_i = \begin{cases} 0 & \text{si pieza } i \text{ no en el centro y sucesora correcta} \\ 1 & \text{si pieza } i \text{ en el centro} \\ 2 & \text{si pieza } i \text{ no en el centro y sucesora incorrecta} \end{cases}$$

h_3 no es admisible, y por lo tanto no es más informado que h_2 , pero es más rápido

- ⊙ El algoritmo A^* resuelve problemas en los que es necesario encontrar la mejor solución
- ⊙ Su coste en espacio y tiempo en el caso medio es mejor que los algoritmos de búsqueda ciega si el heurístico es adecuado
- ⊙ Existen problemas en los que la dimensión del espacio de búsqueda no permite su solución con A^*
- ⊙ Además los nodos a almacenar por A^* crecen exponencialmente si:

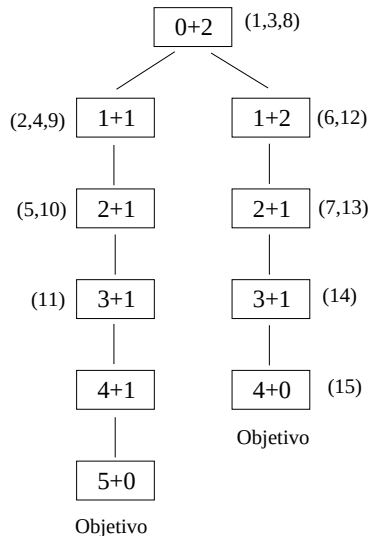
$$|h(n) - h^*(n)| \geq \log(h^*(n))$$

- ⊙ Existen algoritmos que permiten obtener el óptimo limitando la memoria usada:
 - IDA*
 - Best First Recursivo
 - Memory Bound A^* (MA*)

- ⊙ Similar a ID (es decir iteración de búsqueda en profundidad con un límite en la búsqueda)
- ⊙ En ID el límite lo daba una cota máxima en la profundidad
- ⊙ En IDA* el límite lo da una cota máxima sobre el valor de la función f

¡Ojo! La búsqueda es una DFS normal y corriente, el heurístico f sólo se utiliza para podar

Empezamos con $\text{corte} = f(\text{inicial})$



Algoritmo: IDA* (limite entero)

$\text{prof} \leftarrow f(\text{Estado inicial})$

$\text{Actual} \leftarrow \text{Estado inicial}$

mientras no es_final?(Actual) y $\text{prof} < \text{limite}$ hacer

Est_abiertos.inicializa()

Est_abiertos.insertar(Estado inicial)

$\text{Actual} \leftarrow \text{Est_abiertos.primer}()$

mientras no es_final?(Actual) y no Est_abiertos.vacía?() hacer

Est_abiertos.borrar_primer()

Est_cerrados.insertar(Actual)

Hijos $\leftarrow$ generar_sucesores (Actual, prof)

Hijos $\leftarrow$ tratar_repetidos (Hijos, Est_cerrados,
Est_abiertos)

Est_abiertos.insertar(Hijos)

$\text{Actual} \leftarrow \text{Est_abiertos.primer}()$

$\text{prof} \leftarrow \text{prof} + 1$

- ⦿ La función `generar_sucesores` solo genera aquellos con una f menor o igual a la del limite de la iteración
- ⦿ La estructura de abiertos es ahora una pila (búsqueda en profundidad)
- ⦿ Hemos de tener en cuenta que si tratamos los nodos repetidos el ahorro en espacio es nulo

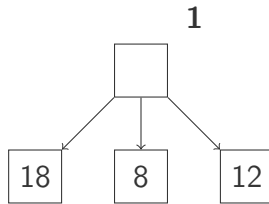
- ⊙ Las reexpansiones de IDA* pueden suponer un elevado coste temporal
- ⊙ Existen algoritmos que por lo general reexpanden menos nodos
- ⊙ Su funcionamiento se basa en eliminar los nodos menos prometedores y guardar información que permita reexpandirlos
- ⊙ Ejemplos:
 - Best first recursivo
 - Memory Bound A* (MA*)

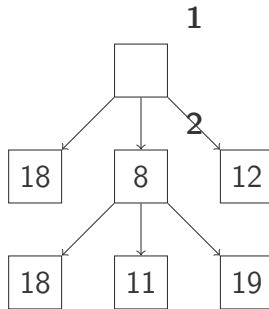
- ⊙ Es una implementación del Best First recursiva con coste lineal en espacio $O(rp)$
- ⊙ **Olvida** una rama cuando su coste supera la mejor alternativa
- ⊙ El coste de la rama olvidada se almacena en el padre como su nuevo coste
- ⊙ La rama es reexpandida si su coste vuelve a ser el mejor (regeneramos toda la rama olvidada)

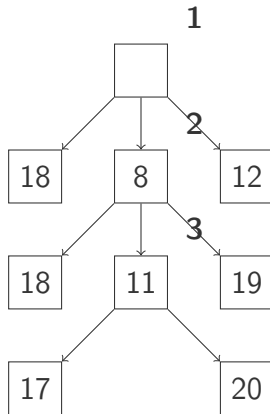
Procedimiento: BFS-recursivo (nodo,c_ alternativo,ref nuevo _coste,ref solucion)

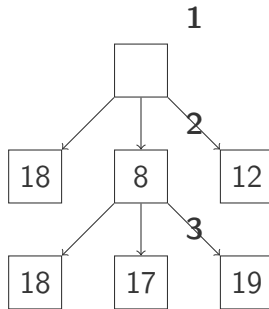
```
si es_solucion?(nodo) entonces
    solucion.añadir(nodo)
sino
    sucesores ← generar_sucesores (nodo)
    si sucesores.vacio?() entonces
        nuevo _coste ←  $+\infty$ ; solucion.vacio()
    sino
        fin ← falso
        mientras no fin hacer
            mejor ← sucesores.mejor _nodo()
            si mejor.coste() > c_ alternativo entonces
                fin ← cierto; solucion.vacio(); nuevo _coste ← mejor.coste()
            sino
                segundo ← sucesores.segundo _mejor _nodo()
                BFS-recursivo(mejor,min(c_ alternativo,segundo.coste()),nuevo _coste, solucion)
                si solucion.vacio?() entonces
                    mejor.coste(nuevo _coste)
                sino
                    solucion.añadir(mejor); fin ← cierto
```

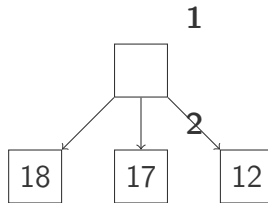


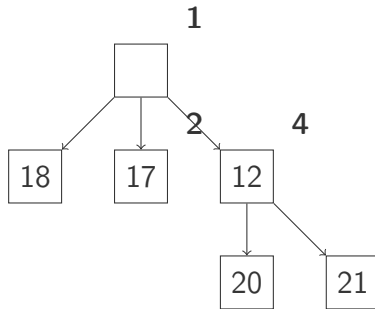


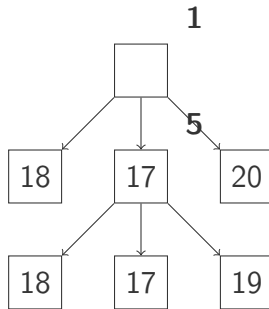


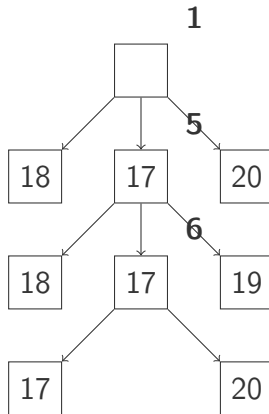


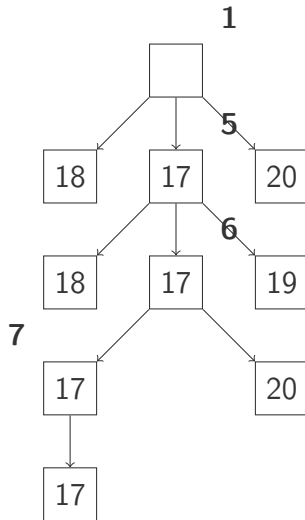


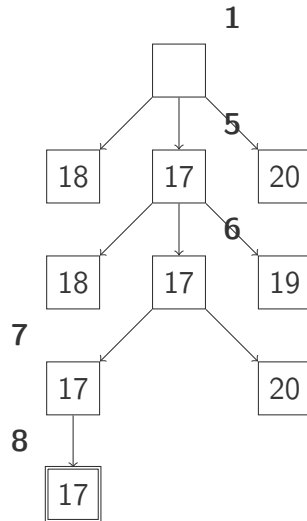












- ⊙ Por lo general reexpande menos nodos que IDA*
- ⊙ El límite de memoria (lineal en espacio) provoca muchas reexpansiones en ciertos problemas
- ⊙ Al no poder controlar los repetidos su coste en tiempo puede elevarse si hay ciclos
- ⊙ Solución: Relajar la restricción de memoria

- ⊙ Impone un límite de memoria (número de nodos que se pueden almacenar, mayor que $O(rp)$)
- ⊙ Exploramos usando A* y almacenamos nodos mientras quepan en la memoria
- ⊙ Cuando no quepan eliminamos los peores guardando el mejor coste de los descendientes olvidados
- ⊙ Reexpandimos si los nodos olvidados son mejores
- ⊙ El algoritmo es completo si el camino solución cabe en memoria